

Intent Engineering in the Claude Code Ecosystem: Implementation Guide

Executive Summary

Intent engineering represents a shift from telling an AI agent *how* to do things to encoding *what it should want* — its objectives, success criteria, constraints, and decision authority. This guide maps that framework onto six components of the Claude Code ecosystem: [CLAUDE.md](#) files, skills, hooks, subagents/agent teams, the `<default_to_action>` pattern, and MCP servers. Each section includes concrete configuration examples drawn from official documentation and community best practices.[1][2]

[CLAUDE.md](#) as an Intent Charter

Beyond Rules: Encoding Objectives

Standard [CLAUDE.md](#) files contain project rules and coding conventions. An intent-engineered [CLAUDE.md](#) goes further by encoding *why* the project exists, what outcomes matter, and how Claude should make autonomous decisions when instructions run out.[2][3]

Claude Code's memory system supports a hierarchy of [CLAUDE.md](#) files — managed policy, user, project, project-local, and auto memory — each loaded into context at session start. The first 200 lines of auto memory's MEMORY.md are injected into the system prompt, and project-level rules in `.claude/rules/` are loaded automatically. This architecture maps directly to the intent engineering framework's layered structure.[4]

Recommended Intent Charter Structure

Structure [CLAUDE.md](#) around three layers: **The What** (tech stack, project structure), **The Why** (architectural decisions, business context), and **The How** (how Claude should work with the project). Intent engineering extends this by adding explicit objectives, health metrics, and decision authority:[3]

CLAUDE.md — Project Intent Charter

Mission (The Why)

Help customers resolve Tier-1 issues quickly without creating more frustration than they started with.

Desired Outcomes

- Customer confirms their issue is resolved
- No follow-up ticket on same topic within 24 hours
- Customer rates interaction as helpful

Health Metrics (Do Not Degrade)

- CSAT must stay above 4.2
- Repeat contact rate must not increase
- Escalation quality score must remain below threshold

Decision Authority

- Autonomous: Refunds under \$50, standard troubleshooting
- Escalate: Legal threats, account cancellations, data access requests
- Never: Delete customer data, override billing without approval

Trade-off Hierarchy

When goals conflict: customer satisfaction > resolution speed > cost efficiency

Project Rules

- @docs/coding-standards.md
- @docs/api-conventions.md

Tech Stack

- Next.js 15, TypeScript, PostgreSQL, Tailwind CSS

The `@import` syntax lets [CLAUDE.md](#) stay lean while referencing detailed documentation. Path-specific rules in `.claude/rules/` can scope intent to particular domains — API endpoints get one set of trade-off priorities, frontend components get another.[4]

Practical Tips

Keep the core [CLAUDE.md](#) under ~150 instructions — Claude Code already injects ~50 instructions in its system prompt, and frontier models handle 150–200 total instructions well before degradation. Use `.claude/rules/` with paths frontmatter to scope domain-specific intent to relevant files. Store personal preferences in `CLAUDE.local.md` (auto-gitignored) and use `~/.claude/CLAUDE.md` for cross-project defaults.[4]

Skills as Intent Specifications

From Instructions to Intent

Claude Code skills are prompt templates packaged as folders with a `SKILL.md` file. When invoked, they inject domain-specific instructions into the conversation context and modify the execution environment (allowed tools, model selection). The skill system uses progressive disclosure — only the description field is loaded into context initially; the full [SKILL.md](#) loads on invocation.[5][4]

The key shift from instruction-based to intent-based skills is moving from imperative steps ("do X when Y") to outcome-oriented framing ("optimize for Z while maintaining constraints A, B, C").

Instruction-Based vs. Intent-Based Skill

Aspect	Instruction-Based	Intent-Based
Framing	"Run linter, fix errors, commit"	"Ensure code meets quality bar; CSAT-equivalent for developers is zero lint warnings and passing tests"
Constraint handling	Hardcoded steps	Explicit constraints + freedom to choose approach
Edge cases	Requires anticipating every scenario	Agent reasons from objectives when instructions run out
Maintenance	Brittle; breaks when workflow changes	Resilient; objectives remain stable as tooling evolves

Intent-Based Skill Example

```

---
name: quality-gate
description: Enforce code quality before commits. Use when the
  user is preparing to commit or has asked for a quality check.
allowed-tools: Bash, Read, Grep, Glob
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "$CLAUDE_PROJECT_DIR/.claude/hooks/no-force-push.sh"
---

```

Quality Gate

Objective

Ensure every commit meets the team's quality bar so reviewers spend time on design, not catching preventable issues.

Desired Outcomes

- Zero lint warnings in changed files
- All existing tests pass; new code has test coverage
- No secrets, API keys, or PII in the diff

Health Metrics

- Build time must not increase by more than 10%
- Do not remove or weaken existing tests

Constraints

- Never force-push to main or release branches
- Never auto-commit without showing the diff first

Approach

Discover what changed, run the appropriate quality checks, fix what you can autonomously, and report what requires human judgment. Use the project's existing tooling (detected from package.json, pyproject.toml, or Makefile).

The hooks frontmatter in the skill definition scopes hard guardrails to the skill's lifecycle — they activate only while this skill runs and are cleaned up afterward.[5][4]

Supporting Files for Progressive Disclosure

Skills support bundled resources: `scripts/` for automation, `references/` for documentation loaded via Read, and `assets/` for templates referenced by path. This maps to intent engineering's progressive context — the [SKILL.md](#) states the objective, and detailed references load only when Claude needs them:[5]

```
quality-gate/
├── SKILL.md
├── scripts/
│   └── detect-toolchain.sh  # Detects lint/test tools
```

```
|— references/
|   |— quality-standards.md # Detailed quality criteria
|— assets/
|   |— report-template.md # Output format template
```

Hooks as Hard Guardrails

Architecture Over Prompts

The intent engineering framework distinguishes between **steering constraints** (prompt-layer guidance that influences reasoning) and **hard constraints** (enforced in orchestration, not prompts). Claude Code hooks map directly to hard constraints — they execute before, during, and after tool calls with deterministic exit codes, independent of what Claude "decides" to do.[2][4]

Claude Code supports 17 hook events spanning the full lifecycle, from SessionStart through PreToolUse, PostToolUse, Stop, SubagentStart/Stop, TeammateIdle, TaskCompleted, and SessionEnd. Hooks come in four types: command (shell scripts), HTTP (external APIs), prompt (LLM-based evaluation), and agent (subagent verification with tool access).[4]

Mapping Intent Boundaries to Hook Events

Intent Boundary	Hook Event	Implementation
"Never delete production data"	PreToolUse (Bash)	Deny commands matching destructive SQL patterns [4]
"Always validate before committing"	Stop	Block stop if tests haven't passed; exit code 2 forces Claude to continue [4]
"Escalate when costs exceed threshold"	PostToolUse	Analyze tool output, inject context about cost limits
"No unauthorized MCP operations"	PreToolUse (mcp__*)	Validate MCP tool calls against an allowlist [4]
"Don't stop until the task list is done"	TaskCompleted	Exit code 2 prevents marking a task as complete [4]
"Subagents must respect scope"	SubagentStart	Log or validate subagent launches match expected types

Intent-Enforcing Hook Examples

Stop Gate — Prevent premature completion:

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "prompt",
            "prompt": "Review the conversation: $ARGUMENTS. Has the user's sta"
          }
        ]
      }
    ]
  }
}
```

```
}  
}
```

Prompt-based hooks use an LLM to evaluate whether to allow the action — a lightweight way to enforce intent alignment without writing deterministic scripts.[4]

Budget Guardian — Enforce cost constraints via MCP:

```
#!/bin/bash  
# .claude/hooks/mcp-cost-guard.sh  
INPUT=$(cat)  
TOOL=$(echo "$INPUT" | jq -r '.tool_name')  
  
# Block expensive MCP operations during non-business hours  
HOUR=$(date +%H)  
if echo "$TOOL" | grep -q "mcp_billing__" && [ "$HOUR" -lt 9 -o "$HOUR" -gt  
    echo "Billing operations restricted to business hours" >&2  
    exit 2  
fi  
  
exit 0
```

PostToolUse Context Injection — Steer after file edits:

```
{  
  "hooks": {  
    "PostToolUse": [  
      {  
        "matcher": "Write|Edit",  
        "hooks": [  
          {  
            "type": "command",  
            "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/remind-intent.sh"  
          }  
        ]  
      }  
    ]  
  }  
}
```



```
]
}
}
```

The `remind-intent.sh` script can read the file that was changed, check if it's in a critical path, and return `additionalContext` reminding Claude of relevant health metrics — a form of runtime intent reinforcement.[4]

Subagents and Agent Teams

Intent Inheritance vs. Intent Specification

Claude Code supports subagents (focused workers that report results back to a parent) and agent teams (independent sessions that coordinate via shared task lists and direct messaging). The intent engineering question is: should each agent have its own intent spec, or inherit from the parent?[6][4]

The answer is **both, layered**. Subagents should inherit the parent's *objective* and *health metrics* but receive their own *scoped constraints* and *decision authority*. Claude Code's architecture supports this — subagents receive only their markdown body as a system prompt (not the parent's full context), plus [CLAUDE.md](#) files.[4]

Subagent Intent Pattern

```
---
name: security-reviewer
description: Review code for security vulnerabilities. Use
  proactively after any code changes touching auth, payments,
  or user data.
tools: Read, Grep, Glob, Bash
disallowedTools: Write, Edit
model: sonnet
memory: project
skills:
  - owasp-top-10
```

```
- api-security-patterns
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/validate-readonly.sh"
---
```

Inherited Objective

You serve the same mission as the parent agent: ship secure, high-quality software that users can trust.

Your Scoped Objective

Identify security vulnerabilities in code changes before they reach production. You are the last line of defense.

Desired Outcomes

- All OWASP Top 10 categories checked against the diff
- Zero false negatives on critical severity issues
- Each finding includes remediation guidance

Decision Authority

- Autonomous: Flag issues, suggest fixes, rate severity
- Escalate: Any finding rated Critical or High
- Never: Modify code (you are read-only)

Health Metrics

- False positive rate must stay reasonable (don't cry wolf)
- Review must complete within 2 minutes

Consult your agent memory for patterns you've seen before.
Update memory with new vulnerability patterns you discover.

Key features: the `memory: project` field enables persistent cross-session learning stored at `.claude/agent-memory/security-reviewer/`. The `skills` field preloads full skill content into the subagent's context at startup. The `disallowedTools` field enforces read-only access

architecturally, and the `PreToolUse` hook adds a second enforcement layer.[4]

Agent Teams for Complex Objectives

Agent teams differ from subagents in communication: teammates message each other directly via a mailbox system and coordinate through a shared task list, rather than routing everything through a parent.[7][6]

For intent engineering, the team lead's role is to decompose the objective into tasks and assign intent-scoped work to teammates. Each teammate gets its own context window and can be configured with different tools, models, and permissions.[6]

Team Lead (your session)

- |— Objective: "Migrate auth module from Express to Fastify"
- |— Health Metrics: All existing tests pass, no API contract changes
- |
- |— Teammate: schema-analyst (Explore agent, read-only)
 - | — Scoped intent: Map all auth endpoints and their contracts
- |
- |— Teammate: implementer (general-purpose, Write + Edit)
 - | — Scoped intent: Rewrite endpoints preserving contracts
- |
- |— Teammate: validator (test-runner agent, Bash)
 - | — Scoped intent: Verify contract compatibility + run tests

Teammates coordinate via `Shift+Up/Down` navigation and `Ctrl+T` for the shared task list. Tasks support dependencies, so the implementer's work auto-unblocks when the schema-analyst completes.[6]

The `<default_to_action>` Pattern

What It Is

This is an official Anthropic prompting pattern for Claude 4.x models, designed to address a specific behavioral trait: when asked to "suggest changes," Claude may literally suggest rather than implement — even when implementation is clearly the user's intent. [8]

The full pattern from Anthropic's prompting best practices:

```
<default_to_action>
```

By default, implement changes rather than only suggesting them. If the user's intent is unclear, infer the most useful likely action and proceed, using tools to discover any missing details instead of guessing.

```
</default_to_action>
```

The conservative counterpart is:

```
<do_not_act_before_instructions>
```

Do not jump into implementation or change files unless clearly instructed to make changes. When uncertain, present options and wait for explicit direction.

```
</do_not_act_before_instructions>
```

How It Works in Practice

Place this in your [CLAUDE.md](#) or in a skill's content. In Claude Code specifically, this pattern interacts with the permission system — Claude still needs permission to write/edit files (unless `permissionMode: "acceptEdits"` is set), so the pattern primarily changes Claude's *intent to act* rather than bypassing safety.[8]

Risks and Mitigations

Risk	Mitigation
Claude modifies wrong files	Use PreToolUse hooks to validate file paths against an allowlist
Cascading unintended changes	Set maxTurns on subagents to limit agentic loop depth [4]
Overengineering	Add explicit steering: "Make minimal changes. A bug fix doesn't need a refactor." [8]
Loss of oversight	Keep permissionMode: "default" so you approve edits; use <default_to_action> only for <i>deciding</i> to act, not for <i>auto-approving</i> actions

The pattern is most powerful when combined with intent engineering's stop rules and health metrics. Claude defaults to action, but hooks enforce boundaries, and health metrics in the prompt steer the aggressiveness of that action.

Recommended [CLAUDE.md](#) Integration

Autonomy Posture

<default_to_action>

Infer intent and implement changes by default rather than just suggesting them. Use tools to discover missing context instead of asking. When multiple approaches exist, choose the one that best serves the stated objective and health metrics.

</default_to_action>

Override Conditions

- Switch to suggestion mode when working in production configs
- Always show the diff before committing
- Never auto-apply database migrations

MCP Servers and Tool Boundaries

Action Schemas as MCP Configuration

Intent engineering's "action schemas" define what tools an agent can access and under what conditions. In Claude Code, this maps to three mechanisms: MCP server scoping, hook-based validation of MCP tool calls, and managed MCP policies.[2][4]

MCP tools appear in Claude Code as `mcp__<server>__<tool>` — for example, `mcp__github__create_issue`. This naming convention enables precise hook matching:[4]

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "mcp__github__delete.*",
        "hooks": [
          {
            "type": "command",
            "command": "echo 'GitHub delete operations require manual approval'"
          }
        ]
      },
      {
        "matcher": "mcp__billing__.*",
        "hooks": [
          {
            "type": "prompt",
            "prompt": "The agent wants to perform a billing operation: $ARGUMENT"
          }
        ]
      }
    ]
  }
}
```

Autonomy Boundaries via MCP Scoping

For subagents, use the `mcpServers` frontmatter field to expose only the MCP servers relevant to that agent's scoped intent:[4]

```
---
name: research-agent
description: Gather information from approved sources
mcpServers:
  - github
  - notion
tools: Read, Grep, Glob
disallowedTools: Write, Edit
---
```

This agent can read from GitHub and Notion but cannot write to either (because Write/Edit are disallowed) and has no access to billing, Slack, or database MCP servers.

Managed MCP for Organizational Intent

Organizations can enforce tool boundaries via `managed-mcp.json`, which takes exclusive control over all MCP servers — users cannot add, modify, or remove servers. Alternatively, `allowedMcpServers` and `deniedMcpServers` in managed settings let users add their own servers within policy constraints:[4]

```
{
  "allowedMcpServers": [
    { "serverName": "github" },
    { "serverName": "notion" },
    { "serverUrl": "https://mcp.company.com/*" }
  ],
  "deniedMcpServers": [
    { "serverUrl": "https://*.untrusted.com/*" }
  ]
}
```

The denylist takes absolute precedence — a server matching any deny entry is blocked even if it appears on the allowlist.[4]

Tool Search for Large Ecosystems

With 106+ skills and multiple MCP servers, tool definitions can overwhelm the context window. Claude Code's MCP Tool Search (enabled by default when tools exceed 10% of context) dynamically loads tools on demand rather than preloading all of them. This means the intent charter in [CLAUDE.md](#) becomes even more important — it guides Claude's tool *discovery* reasoning, not just tool *use* reasoning. [4]

Putting It All Together: The Intent Stack

The complete intent engineering implementation in Claude Code forms a layered architecture:

Layer	Claude Code Component	Intent Engineering Role
Objective	CLAUDE.md (mission, outcomes, health metrics)	Defines what the system should want
Steering	Skills (SKILL.md with outcome framing)	Guides approach and trade-offs
Hard Guardrails	Hooks (PreToolUse, Stop, TaskCompleted)	Enforces constraints architecturally
Scoped Autonomy	Subagents (tools, permissions, memory)	Distributes decision authority
Coordination	Agent Teams (task lists, mailbox)	Enables multi-agent objective decomposition
Action Posture	<default_to_action> in CLAUDE.md	Sets the default bias toward implementation
Tool Boundaries	MCP config + managed policies	Constrains the action space

The critical insight is that prompt-layer steering ([CLAUDE.md](#), skills) and architectural enforcement (hooks, tool restrictions, MCP policies) serve different roles. Health metrics *steer*; guardrails *enforce*. An intent-engineered Claude Code setup uses both layers together, so the agent has clear objectives to reason from *and* hard boundaries it cannot cross regardless of its reasoning.[2]

References

1. [Intent Engineering: Why the Future of Prompting Isn't ... - Prompt Sigma](#) - Intent Engineering recognizes that modern LLMs are no longer passive recipients of instructions; the...
2. [The Intent Engineering Framework for AI Agents](#) - The Intent Engineering Framework shows how to structure objectives, outcomes, health metrics, constr...
3. [Claude Code Tutorial for Beginners - Complete 2026 Guide to AI ...](#) - Structure your [CLAUDE.md](#) around three layers: The What — Your tech stack, project structure, key pac...
4. [Agent Skills - Claude Code](#) - Skills can include multiple files in their directory. This keeps [SKILL.md](#) focused on the essentials ...
5. [Claude Agent Skills: A First Principles Deep Dive - Han Lee](#) - Skills are defined as folders that include instructions, scripts, and resources that Claude can load...
6. [Orchestrate teams of Claude Code sessions](#) - Agent teams let you coordinate multiple Claude Code instances working together. One session acts as ...
7. [Claude Code Agent Teams: The Complete Guide 2026](#) - Complete guide to Claude Code Agent Teams for parallel multi-agent development. What they are, when ...
8. [Skill authoring best practices - Claude Docs](#) - This guide provides practical authoring decisions to help you write Skills that Claude can discover ...